

Verify Before You Execute: A Hybrid Symbolic–Learned Pre-Execution Verifier for LLM-Generated Plans

Ari Bajaj

Illinois Mathematics and Science Academy

abajaj1@imsa.edu

Abstract

LLM agents increasingly emit multi-step plans—action sequences, tool-call chains—that downstream systems execute directly. Executing a flawed plan is costly, yet the dominant safeguards are either an LLM judging its own kind or no check at all. We present a pre-execution verifier that translates a free-text plan into a typed action schema, *soundly* checks consistency, goal completeness, and resource feasibility by forward simulation, and fuses the symbolic verdict with a small calibrated *trust model* that estimates whether the NL→schema translation itself was faithful. Hard symbolic failures always reject; the trust score ranks residual translation risk. On three domains (Blocksworld, Logistics, and a tool-use environment that reused the pipeline with zero code changes), against oracle labels validated by two independent simulators agreeing on 360/360 records, the hybrid attains F1=0.949 at perfect recall and—most importantly—raises downstream execute-or-reject task success from 61.1% (no verifier) to 93.1%, beating an LLM-judge gate (87.5%) because its symbolic explanations make replanning feedback actionable. We report three negative-space findings with equal prominence: properly-budgeted LLM judges are *perfect* detectors on short-horizon plans; a constrained output format makes the translation lossless, eliminating the trust model’s target class; and a naive token budget silently turned a fail-open judge baseline into “no verifier” on 100% of records. Code, data generators, and all results are reproducible from a single make target on a laptop.

1 Introduction

A growing class of systems lets a large language model (LLM) propose a multi-step plan—a sequence of robot actions, API calls, or tool invocations—and then executes that plan with little or no intermediate checking. When the plan is flawed, the cost is paid at execution time: side-effecting API calls fire, budgets are spent, and physical or irreversible actions occur before the flaw surfaces. Benchmarks consistently find that LLM plans are unreliable in exactly the ways that matter: preconditions are violated, goal conjuncts are silently dropped,

and resource limits are exceeded [12, 5].

The natural safeguards occupy two unsatisfying extremes. Purely symbolic validation in the style of VAL [3] is sound but assumes the plan already exists in a formal language—while LLM plans arrive as free text, and the lossy translation from text to formalism becomes an unmodeled failure source. Purely learned safeguards—asking an LLM judge “is this plan valid?” [14]—inherit the fallibility they are meant to check, and recent evidence suggests LLMs are poor self-verifiers [4, 11].

We study the middle: a **hybrid pre-execution verifier** for LLM plans. An LLM extractor translates each free-text step into a typed action schema; a sound symbolic checker forward-simulates the extracted plan and verdicts three independent dimensions (action consistency, goal completeness, resource feasibility); and a small calibrated *trust model*—trained on extraction confidence, k -resample self-consistency agreement, and plan-shape features—estimates the probability that the symbolic verdict is *faithful* to the plan as written, i.e. that the translation did not silently change what is being checked. The fusion rule is asymmetric by design: a hard symbolic failure always rejects, regardless of trust; among symbolic passes, low trust flags residual risk.

Contributions. (1) A pipeline architecture and open implementation for sound pre-execution verification of free-text LLM plans, with a learned estimate of translation faithfulness as a first-class signal. (2) A synthetic-data methodology with an *independent* oracle: two separately implemented simulators (labeler and verifier) whose agreement on every record (360/360; 1,440/1,440 per-dimension comparisons) anchors all reported numbers. (3) A generality test: a tool-use domain (auth scopes, prerequisite calls, quotas, spend budgets) was expressed in the same DSL and ran through the entire pipeline with *zero* code changes. (4) A downstream execute-or-reject experiment showing the hybrid’s real edge is not detection accuracy but *replan feedback quality*: symbolic explanations convert rejections into successful second attempts more often than judge critiques (5 vs. 9 dead-ends). (5) Three honestly-reported negative results that materially reframe the design space for verifier research, including a deployment failure mode in which a fail-open LLM judge silently degrades to no

verification at all.

2 Related Work

LLM planning and its critics. PlanBench and successors document systematic planning failures in LLMs [12]. The LLM-Modulo position argues LLMs should generate while external sound modules verify [5]; LLM+P [6] translates problems into PDDL and delegates to a classical planner. We keep the LLM as planner and verify its output, targeting settings (e.g. tool use) where a complete classical model exists only implicitly.

Plan validation. VAL [3] validates PDDL2.1 [1] plans against a formal domain. Our symbolic core is a deliberately small reimplementation of this idea over a typed DSL with per-step resource accounting; the novelty is not the checker but its coupling to a learned model of *whether the checker saw the right plan*.

LLM self-checking and repair. LLM-as-judge [14], Self-Refine [7], and Reflexion [10] use LLMs to critique or repair their own outputs; Huang et al. [4] and Stechly et al. [11] find intrinsic self-correction weak on reasoning and planning. Our baselines instantiate both judge variants and a Reflexion-style repairer on identical inputs, and our downstream experiment measures what each safeguard is worth at execution time.

Calibration and self-consistency. We use k -resample agreement in the spirit of self-consistency [13] as a *feature* rather than a decoding rule, Platt scaling [9] for calibration, and expected calibration error [8, 2] for evaluation.

3 Method

3.1 Plan and Domain Representation

A domain is a set of typed predicate definitions, action schemas (preconditions, add/delete effects), and *resource dimensions*—named scalars with an initial value, a floor, and a cap, adjusted by per-action deltas. A problem grounds a domain with typed objects, an initial state, and a conjunctive goal. Validation is structural: an action schema may only reference declared predicates and resources with matching arity and types. This DSL is intentionally small—no disjunctive preconditions, conditional effects, or typed return values—but Section 5.6 shows it stretches to tool-use constraints without modification.

3.2 Symbolic Verifier

Given a sequence of ground actions, the verifier forward-simulates from the initial state and emits a three-part verdict with a machine-readable explanation: **consistency** (every

step’s action exists, is well-typed, and its preconditions hold when executed in order), **goal completeness** (every goal conjunct holds in the final state), and **resource feasibility** (every prefix of the running sum respects every dimension’s floor and cap). Progression is *lenient*: a violated precondition is recorded but effects still apply, so one early slip cannot mask an independent later flaw—each dimension verdicts independently. Upstream parse or extraction failures are injected as consistency violations, since an executor could not run those steps. `overall_valid` is the conjunction of the three checks.

3.3 NL→Schema Extraction with Self-Consistency

Free-text steps are mapped to ground actions by an LLM extractor using the API’s structured-output mode, validated against the domain (action name, arity, object types). On validation failure the extractor retries once with the error as feedback, then falls back to a null extraction with confidence 0—it never crashes, it degrades. Each step is resampled $k=3$ times at temperature 0.7; the modal extraction is used, and two agreement statistics (exact-match fraction and action-type-match fraction) are retained. These, with the extractor’s self-reported confidence, become features rather than being discarded.

3.4 Trust Model and Fusion

The trust model answers: *given how the extraction went, how likely is the symbolic verdict faithful to the plan as written?* Its label is agreement between the production verdict (LLM extraction path) and a reference verdict computed by a deterministic rule-based parse of the original plan text. Features (17) span extraction confidence (mean/min), k -resample agreement, plan-shape statistics, the fraction of failed-extraction steps, a TF-IDF character- n -gram similarity between the preconditions the extractor *mentioned* and those the schema *requires*, the symbolic verdict bits and violation counts, and a domain indicator. The classifier is a class-balanced logistic regression, Platt-scaled on a held-out validation split; all splits are by *problem*, so no problem’s records straddle train/test.

Fusion is a two-clause rule with a tested invariant:

$$\text{reject} \iff \neg \text{symbolic_valid} \vee \text{trust} < \tau,$$

where the first clause is unconditional—no trust score can rescue a hard symbolic failure—and τ is picked on validation. $\tau=0$ recovers symbolic-only exactly.

4 Experimental Setup

4.1 Domains, Problems, and Gold Plans

Blocksworld (4 operators + a crane-energy resource), **Logistics** (trucks/planes over two cities, fuel + budget), and **Tools** (Section 5.6). A seeded generator produces problems whose

BFS-optimal gold plans span 3–8 steps; determinism is enforced cross-process (a PYTHONHASHSEED ordering bug found by a subprocess test motivated sorting all set-derived output). Resource caps are tightened to $\lceil 1.25 \times \rceil$ the gold plan’s consumption so that wasteful plans can actually overrun.

4.2 Plan Generation with Failure Injection

For each problem, `claude-haiku-4-5` (temperature 1.0) writes a plan under four prompt conditions: *baseline* (honest prompt), *goal-omission* (one goal conjunct silently dropped from the prompt; labels use the full goal), *resource-blind* (limits stripped), and *distractor* (plausible invalid actions offered). A pilot calibration confirmed the conditions amplify naturally occurring failure modes rather than inject alien ones: 43–47% of *baseline* plans are already flawed, and goal-omission roughly quadruples goal-incompleteness. Dataset: 3 domains $\times$ 30 problems $\times$ 4 conditions = 360 records.

4.3 Independent Oracle and Soundness Evidence

Ground-truth labels come from an oracle labeler that is a *separate implementation* of plan simulation from the verifier (shared design semantics, disjoint code). Agreement between the two on rule-parsed plans is therefore evidence, not tautology: across all 360 records they agree on overall validity 360/360 and on all 1,440 per-dimension comparisons, with every flaw type additionally covered by hand-verified adversarial fixtures. The evaluation CLI recomputes this cross-check on every run and fails loudly on any disagreement.

4.4 The Prose Regime

Our first finding is methodological: with a constrained output format (Step *N*: `action(args)`), the LLM extraction path’s verdict agreed with the rule-based reference on **240/240** records—the NL→schema translation was lossless, the trust label had one class, and the learned layer had nothing to model. This is a genuine scoping result: *where plans can be forced into a rigid format, a rule parser plus a sound checker suffices*. To study the setting the trust model exists for, the production pipeline verifies an LLM-generated *prose paraphrase* of each plan (one sentence per step, no rigid syntax), while labels stay anchored to the original constrained text. In the prose regime 13/360 records (3.6%) get unfaithful verdicts—sparse, but a real target class.

4.5 Baselines

On identical prose inputs: **LLM judge** (zero-shot and chain-of-thought, verdict parsed from a required VERDICT: line, fail-open on unparseable output, mirroring naive deployment); **symbolic-only** ($\tau=0$); **learned-only** (the trust feature vector with all five symbolic-verdict features removed, trained to

System	P	R	F1
Hybrid (ours, τ picked on val)	0.903	1.000	0.949
Symbolic-only	0.903	1.000	0.949
Learned-only (no symbolic feats)	0.618	0.750	0.677
LLM judge, zero-shot	1.000	1.000	1.000
LLM judge, CoT	1.000	1.000	1.000
Self-repair (change detection)	0.966	1.000	0.982

Table 1: Detection on the 72-record pooled test split (positive class = flawed). Per-flaw recall for the hybrid: 20/20 inconsistency, 10/10 goal-incompleteness, 5/5 resource-infeasibility.

predict validity directly—the no-grounding ablation); and **self-repair** (Reflexion-style single-pass critique+rewrite [10], with change-detection standing in for a verdict).

4.6 Metrics

Positive class = *flawed plan*; rejecting is a positive prediction. We report P/R/F1 overall, per domain, and per flaw type; ECE with reliability diagrams for the trust model; API-call and local-latency cost proxies; and the downstream experiment of Section 5.3. Test split: 72 records (problem-level, 216/72/72).

5 Results

5.1 Detection

Table 1 contains a result we report at full prominence: **both LLM judges are perfect on this test split**. Plans of 3–8 steps are short enough for the judge model to simulate flawlessly when given an adequate token budget (see Section 5.4). At this horizon, “hybrid beats LLM-judge on detection” is *not supported*; the hybrid’s measured advantages lie elsewhere (downstream feedback, cost, determinism, machine-readable explanations), and whether detection separates at longer horizons is the central full-scale question. The learned-only ablation (F1 0.677) confirms the symbolic grounding carries the detection signal.

The hybrid’s only errors are 3 false rejects, all on the Tools domain (precision there 0.833, perfect elsewhere), all with one signature: the extractor mangles hyphenated arguments (`flights-scope` → `flights`), producing unknown-object violations on genuinely valid plans.

5.2 The Trust Signal Points the Right Way—but the Fusion Rule Cannot Use It

The validation-picked threshold is $\tau=0$: the trust layer never changes a decision at dev scale, and the test split contains zero symbolically-clean-but-flawed records for it to catch (41 symbolic passes, all genuinely valid). Yet the signal itself is demonstrably real: the three false-rejected mistranslations receive the trust model’s *lowest* scores in the entire test split

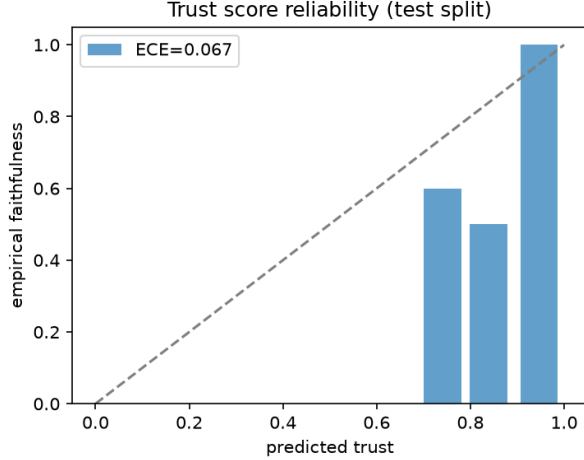


Figure 1: Trust-model reliability diagram (test split). Mass concentrates in the top bin at accuracy 1.0; the sparse low-confidence bins are exactly the mistranslated records of Section 5.2.

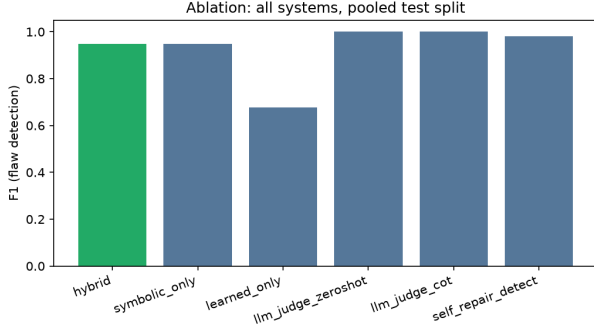


Figure 2: F1 by system and domain. The learned-only bar shows the cost of removing symbolic grounding; judge bars are at ceiling at this plan length.

(0.74–0.82, vs. >0.9 elsewhere; classifier accuracy 0.958, ECE 0.067, Figure 1). The model detects unfaithful translation exactly as designed—but our fusion rule, like prior gate-then-execute designs, only lets trust *add* rejections. A bidirectional rule (low-trust symbolic reject $\rightarrow$ re-extract, or fall back to the rule parser) would have repaired all three errors and is the concrete architectural lesson of this study.

5.3 Downstream: Execute or Reject

Table 2 is where the architecture earns its keep. Both gates eliminate flawed execution entirely, but the hybrid converts more rejections into successful second attempts: its replan feedback is the symbolic explanation itself (“step 3: unknown object ...”, “goal conjunct on(b3,b2) unmet”), which is more actionable than the judge’s free-text critique—5 vs. 9 dead-ends, 93.1% vs. 87.5% final task success. Detection parity, feedback superiority.

Arm	Success	Flawed exec.	Replans	Dead ends
No verifier	0.611	0.389	0	0
Hybrid-gated	0.931	0.000	31	5
CoT-judge-gated	0.875	0.000	28	9

Table 2: Execute-or-reject with one feedback-driven replan (72 test records). Success = an executed plan that is actually valid.

5.4 A Deployment Failure Mode: the Silently Absent Judge

Our first zero-shot judge used a 64-token budget, reasoning that a one-line verdict needs no more. The judge model ignored the “respond with exactly one line” instruction, simulated the plan step by step, and was truncated before ever emitting its verdict—on **360/360 records**. Because unparseable output fails open (as a naive deployment would), the judge scored $P=R=0.0$ while appearing to run normally: the system silently became *no verifier at all*. At 512 tokens, 41% of outputs still truncated; only at 2,048 (the CoT budget) did parse failures reach zero—whereupon the judge jumped from 0.0 to a perfect 1.0. We fixed the baseline and report the fixed numbers, but the episode is itself a finding: fail-open verdict parsing plus format non-compliance is a silent total-failure mode, and the zs-vs-CoT contrast in any judge comparison is confounded by budget unless equalized.

5.5 Self-Repair and Cost

The Reflexion-style repairer is a surprisingly strong *fixer*—23/28 flawed plans repaired, 0/44 valid plans broken—but an unreliable *detector* (change-detection conflates cosmetic rewrites with flaw detection). Cost: the symbolic path decides in $\sim 0.2 \mu s$ locally with zero API calls; the hybrid adds $k \times$ steps extraction calls (~ 12 –24, batchable); each judge call generates up to 2,048 tokens. A verifier that is free at decision time changes how often one can afford to check.

5.6 Generality: the Tool-Use Domain

The Tools domain models a travel-booking agent: eight tools, auth scopes as predicates, prerequisite calls threaded as boolean return-value facts (`search.flights` adds `flight-options(trip)`, which `book.flight` requires), an API quota ($-1/\text{call}$) and a spend budget (flight 300, hotel 200) as resource dimensions. **No changes to any pipeline code were required**—the same labeler, verifier, extractor, and trust model ran as-is, and the domain’s verdicts cross-check 120/120 against the oracle. The one representational wrinkle: the DSL lacks constant action arguments, so scopes are pinned by identity predicates. Notably, the pipeline’s only false rejects arise here, from hyphenated tool-argument extraction—evidence that tool-style vocabularies stress the NL $\rightarrow$ schema seam more than classical planning vocabularies do.

6 Limitations

All numbers are dev-scale: 360 records, a 72-record test split, 13 negative trust labels, one model family (claude-haiku-4-5) for every LLM role, single seed. The judges’ perfect detection means the full-scale run (longer horizons, stronger and weaker judge models, multiple seeds) is not optional but the crux. The trust layer’s value proposition is confined to the prose regime by construction (Section 4.4); the paraphrase step can itself add label noise (all systems see identical text, so comparisons stay like-for-like). The oracle and verifier share design semantics though not code, so a shared semantic error would evade the cross-check; hand-verified fixtures and (future) VAL [3] validation are the defenses. Precondition similarity is lexical; the DSL omits conditional effects and typed return values; the downstream replan arm is gated by the sound rule-based check, so its “accepted implies valid” property is by construction and the informative quantities are replan success and ungated flawed-execution cost.

7 Conclusion

We built and honestly evaluated a hybrid pre-execution verifier for free-text LLM plans. At short horizons its detection matches a sound symbolic check and is beaten to ceiling by a well-budgeted LLM judge—but it wins where execution happens: zero flawed executions, the highest downstream task success (93.1%), and replan feedback that turns rejections into successes. Three findings reframe the design space: constrained formats make translation lossless (making the trust problem vanish where they are available); trust signals should flow *both* ways through the fusion rule, triggering re-extraction rather than only extra rejection; and fail-open judge deployments can silently become no verifier at all. The full-scale question—whether judge detection degrades at horizons where sound simulation does not—is precisely what this laptop-scale infrastructure now makes cheap to ask.

Reproducibility

All code, domain generators, prompts, and evaluation scripts are in the project repository; `make reproduce` regenerates every number and figure in this paper from fixed seeds (`make reproduce FULL=1` for the full-scale configuration). Datasets are synthesized locally; the only external dependency is an Anthropic API key.

References

- [1] Maria Fox and Derek Long. PDDL2.1: An extension to PDDL for expressing temporal planning domains. *Journal of Artificial Intelligence Research*, 20:61–124, 2003.
- [2] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.
- [3] Richard Howey, Derek Long, and Maria Fox. VAL: Automatic plan validation, continuous effects and mixed initiative planning using PDDL. In *16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, pages 294–301, 2004.
- [4] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- [5] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. LLMs can’t plan, but can help planning in LLM-modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [6] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. LLM+P: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.
- [7] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the 29th AAAI Conference on Artificial Intelligence*, 2015.
- [9] John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*. MIT Press, 1999.
- [10] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [11] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. In *International Conference on Learning Representations (ICLR)*, 2025.
- [12] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. PlanBench: An extensible benchmark for evaluating large lan-

guage models on planning and reasoning about change. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

- [13] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [14] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.